Branch and Bound: 0/1 Knapsack Problem

ANUSKA NATH

002311001003

Information Technology - UG3

What is the 0/1 Knapsack Problem?

Imagine you're a thief with a limited-capacity knapsack trying to maximize profit from stolen items. You can either take an item whole or leave it—no splitting allowed.

This classic optimization problem has practical applications in resource allocation, investment decisions, and cargo loading.

Problem Statement

*We have a knapsack with **15 kg** capacity and 4 items to choose from:*

Item	*1*	*2*	*3*	*4*
Profit (p_i)	*10*	*10*	*12*	*18*
Weight (w_i)	*2*	*4*	*6*	*9*

***Constraint:** Total weight ≤ 15 kg. **Goal:** Maximize total profit.*

Why Branch and Bound?

Branching

Explore all possible combinations by creating a state space tree where each node represents a decision.

Bounding

Calculate upper bounds to eliminate unpromising branches early, saving computation time.

Optimality

Guarantees finding the globally optimal solution, not just a local maximum.

The Solution Process

01

Initialize State Space Tree

Create nodes representing inclusion/exclusion decisions for each item.

02

Calculate Upper Bounds

Use fractional knapsack relaxation to estimate maximum possible profit.

03

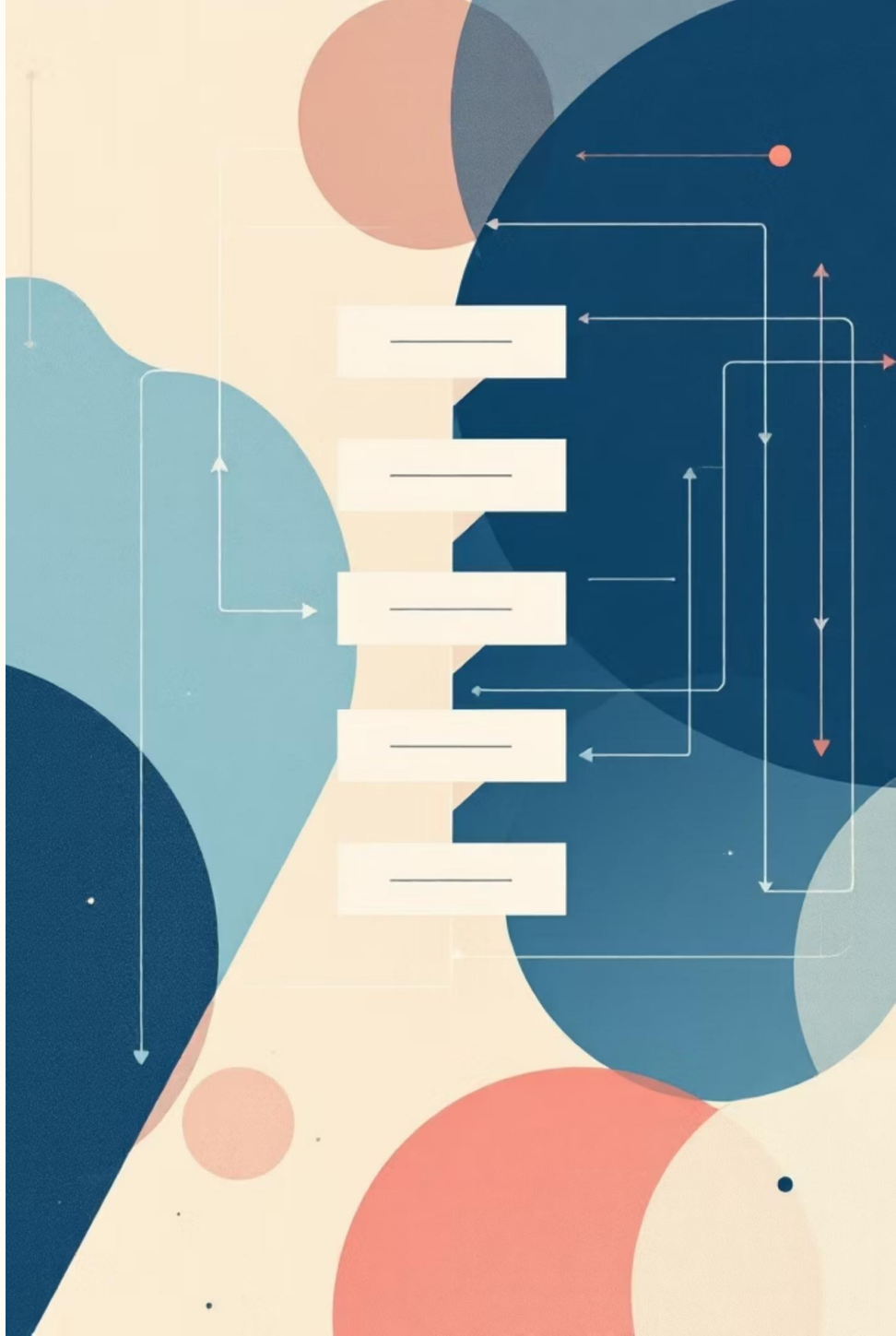
Prune Dead Nodes

Eliminate branches where upper bound is less than current best solution.

04

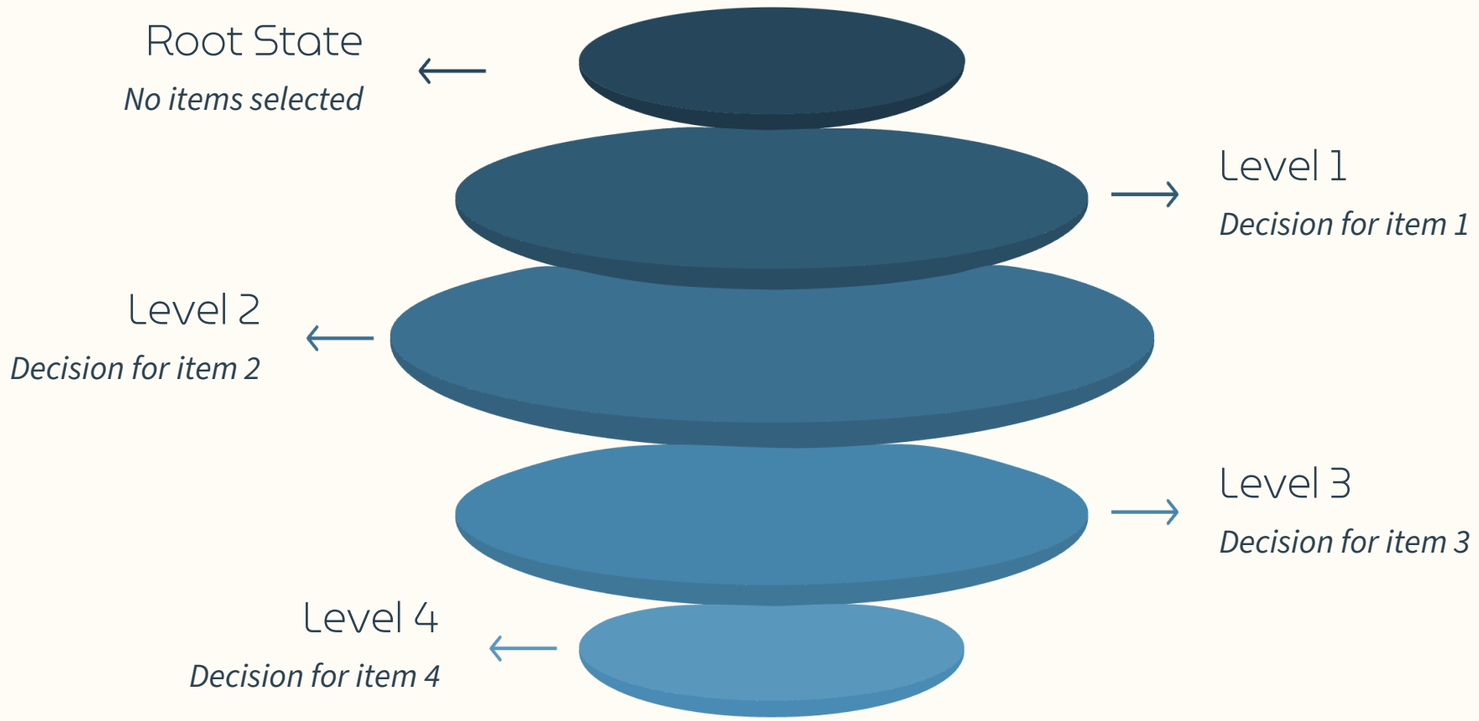
Track Best Solution

Maintain maximum profit found and corresponding item selection.



Step 1: Building the State Space Tree

Each level represents an item decision. Left branch = include item, Right branch = exclude item.



Tree Structure

- *Root: No items selected*
- *Level 1: Item 1 decision*
- *Level 2: Item 2 decision*
- *Level 3: Item 3 decision*
- *Level 4: Item 4 decision*

Node Information

Each node stores: current profit, current weight, and upper bound estimate.

Step 2: Upper Bound Calculation

Fractional Knapsack Relaxation

Temporarily allow fractional items to estimate maximum possible profit from current node.

1. *Sort remaining items by profit-to-weight ratio*
2. *Add items fully until capacity limit*
3. *Add fraction of next item to fill remaining space*

Profit/Weight Ratios

Item	*Ratio*
1	*10/2 = 5.0*
2	*10/4 = 2.5*
3	*12/6 = 2.0*
4	*18/9 = 2.0*

Example: calculation for root node

- *Root node: capacity = 15*
- *Take item 1 fully: profit = 10, weight = 2*
- *Take item 2 fully: profit = 20, weight = 6*
- *Take item 3 fully: profit = 32, weight = 12*
- *Remaining capacity = 3*
- *Take fraction of item 4 profit += 18 × (3/9)*
- *Upper Bound = 38*

Step 3: Traversing and Pruning

Start at Root

Current profit = 0, Weight = 0, Upper bound = -38 (all items fractional)

Explore Item 1 Branch

Include item 1: Profit = 10, Weight = 2, Upper bound = -38

Continue Deepening

Calculate bounds at each node. If bound > current best, prune branch. Always expand the node with the highest upper bound.

Backtrack When Needed

When reaching leaf or dead end, backtrack to explore alternative paths.

Step 3.5: Calculating Node Values - Example

*We analyze Node 6 (x1=1, x2=1) to determine its potential. Since we are branching, we use the fractional knapsack method to find the **Upper Bound (UB)**, representing the maximum possible profit achievable from this point forward.*

1

Initial State

Current Weight: 6

Current Profit: 20

Capacity Remaining: 9

Items taken: 1 & 2

2

Add Item 3

Weight added: 6

Profit added: 12

New Total Weight: 12

New Total Profit: 32

3

Add Item 4 (Fraction)

Remaining capacity: 3

Item 4 value: 18 (weight 9)

Fractional profit: 18 × (3/9) = 6

4

Final Result

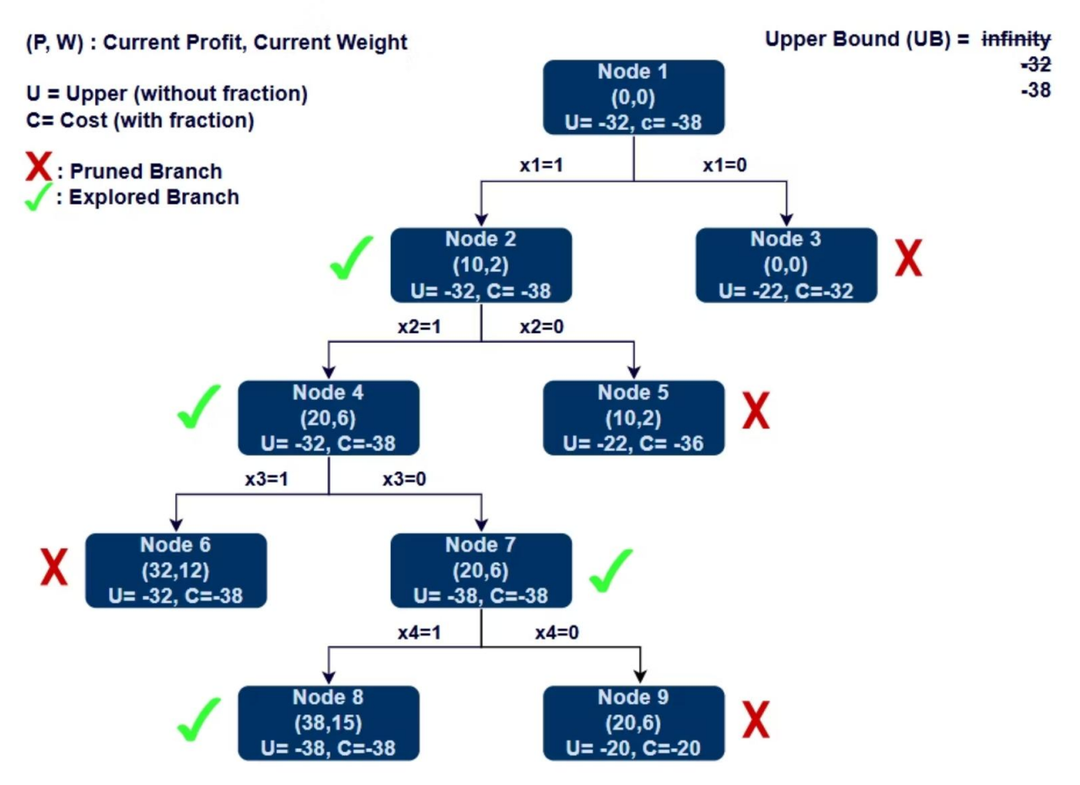
***Upper (U) =** -(profit without fraction) = -32*

***Cost (C) =** -(32+6) = -38*

***Why use fractional items?** The fractional knapsack calculation provides an optimistic upper bound. It ensures that if the best possible scenario (UB) is worse than our current best solution, we can safely prune this entire branch.*

Visualizing the State Space Tree

The Branch and Bound algorithm explores the state space tree by calculating upper bounds (UB) for each node. If a node's UB is less than or equal to the current best profit, we prune that branch, avoiding unnecessary calculations.



For each item i : xi = 0 means item not included, xi=1 means item included.

Pruning Logic

We prune branches where the potential profit cannot exceed our current best:

- ***Node 3 (x1=0):*** *U= -22 >= UB(-32). Pruned.*
- ***Node 5 (x2=0):*** *U= -22 >= UB (-32). Pruned.*
- ***Node 6 (x3=1):*** *U= -32 >= UB (-38). Pruned. [UB is updated to -38 here because C=-38].*
- ***Node 9 (x4=0):*** *U= -20 >= UB (-38). Pruned. [Nothing more to explore]*

Legend

*UB = **Upper Bound** (most negative value of U found)*

- *if **U>=UB** : branch pruned*
- *if **U < UB** : UB = U, branch explored.*

Step 4: The Optimal Solution

38

Maximum Profit

Achieved by optimal item selection

15

Total Weight

Within 15 kg capacity limit

3

Items Included

Specific combination yielding max profit

Selected Items for Optimal Solution

- *Item 1 (profit: 10, weight: 2)*
- *Item 2 (profit: 10, weight: 4)*
- *Item 4 (profit: 18, weight: 9)*

Solution Vector

1	*2*	*3*	*4*
1	*1*	*0*	*1*

Total: *10 + 10 + 18 =* ***38 profit****, 2 + 4 + 9 =* ***15 kg*** *(at capacity limit)*

Key Takeaways

Efficiency Through Pruning

Branch and bound avoids exploring all 2^n possibilities by eliminating unpromising branches early using upper bounds.

Guaranteed Optimality

Unlike greedy approaches, this method guarantees finding the globally optimal solution for 0/1 knapsack problems.

Practical Applications

Used in resource allocation, project selection, cargo loading, and any scenario requiring optimal subset selection under constraints.